

IF3170 Inteligensi Artifisial

Institut Teknologi Bandung

Linear Regression & Logistic Regression

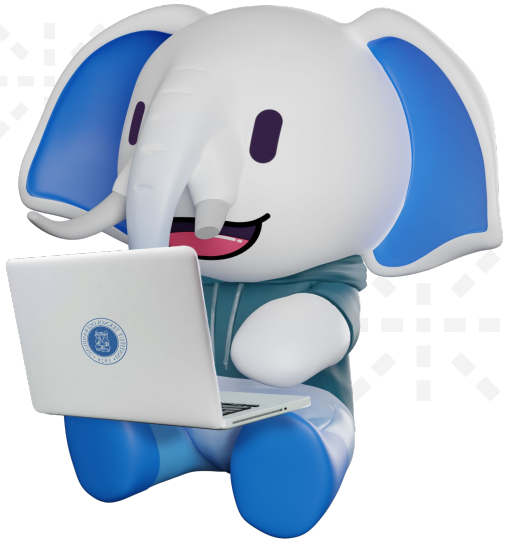
Semester I TA 2026/2027

Outline

- Review:
    - Learning Agent
    - Introduction to Data Science
    - CRISP-DM
    - Data Understanding
    - Data Preparation
    - Decision Tree Learning
    - K-Nearest Neighbor
- Linear Regression
- Logistic Regression

Institut Teknologi Bandung www.itb.ac.id @itb1920 @itbofficial

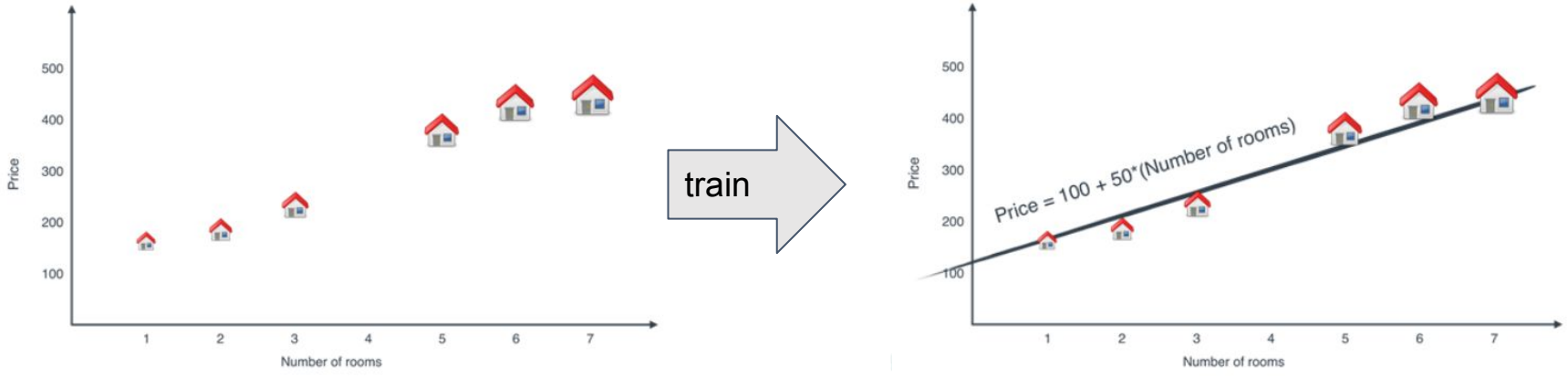
Linear Regression



Institut Teknologi Bandung www.itb.ac.id @itb1920 @itbofficial

Regression: Introduction

As an example, we are a real estate agent, and in charge of selling a new house. We don't know the price, and we want to infer it by comparing it with other houses. We look at features of the house which could influence the house, i.e. number of rooms. At the end of the day, what we want is a formula on the feature which gives us the price of the house, or at least an estimate for it.



https://livebook.manning.com/book/grokking-machine-learning/3-1-the-problem-we-need-to-predict-the-price-of-a-house/v-4/44

Linear Relationship

- The price of the house is dependent variable or response, and number of rooms is independent variable or regressor or predictor.
- A reasonable form of a relationship between the response Y and the regressor x is the linear relationship: $Y = \beta_0 + \beta_1 x$

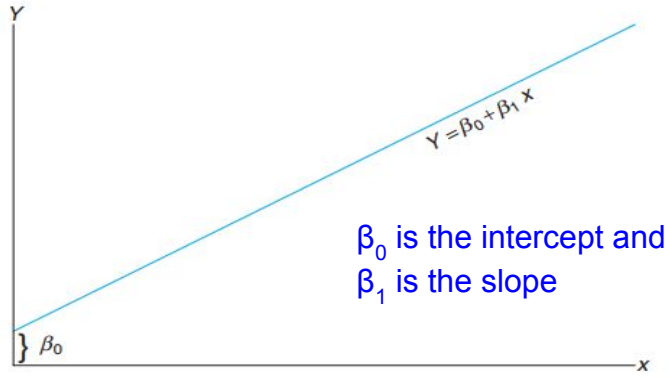
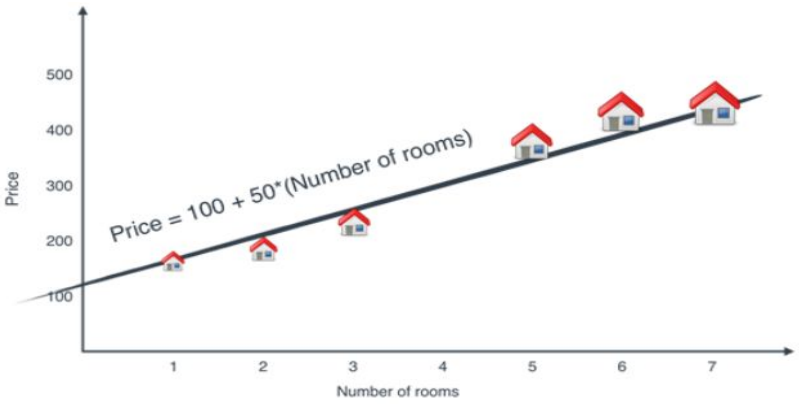


Figure 11.1: A linear relationship; β_0: intercept; β_1: slope.

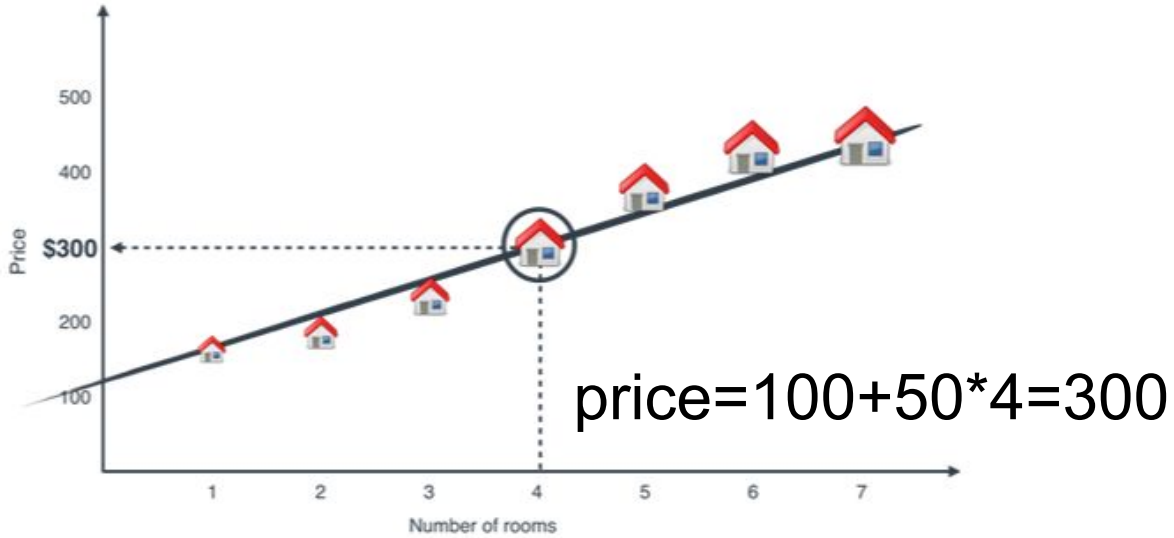


https://livebook.manning.com/book/grokking-machine-learning/3-1-the-problem-we-need-to-predict-the-price-of-a-house/v-4/44

Institut Teknologi Bandung www.itb.ac.id @itb1920 @itbofficial

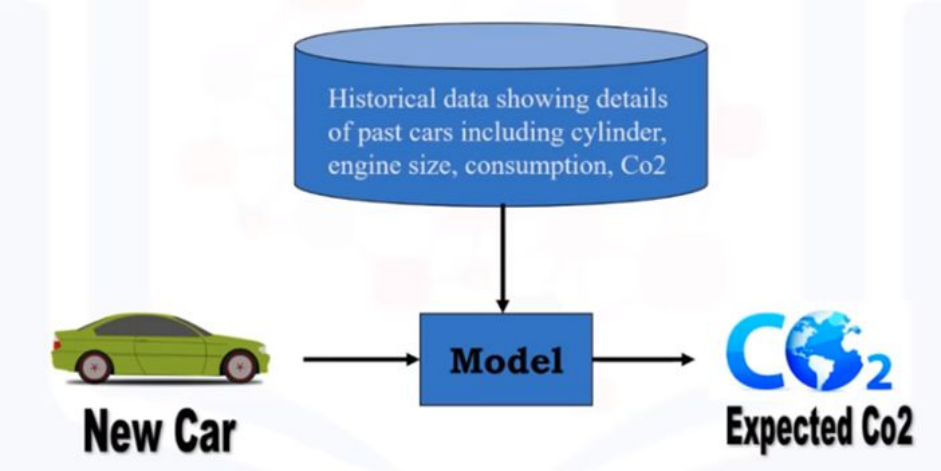
Regression: Predict Response Y based on Regressor x

Figure 6. Our task is now to predict the price of the house with 4 rooms. Using the model (line), we deduce that the predicted price of this house is $300.



https://livebook.manning.com/book/grokking-machine-learning/3-1-the-problem-we-need-to-predict-the-price-of-a-house/v-4/44

Regression Modeling: Example

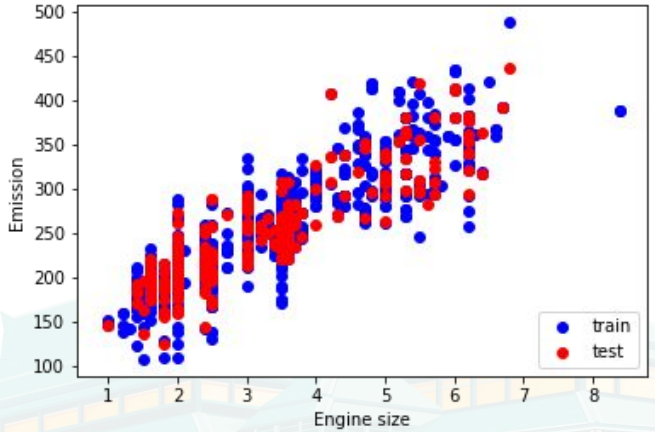



```
1 df[['ENGINESIZE','CO2EMISSIONS']].sample(5)
```

	ENGINESIZE	CO2EMISSIONS
697	2.0	196
872	3.8	253
614	2.4	200
351	5.4	382
426	3.5	258

```
1 df.shape
```

```
(1067, 13)
```



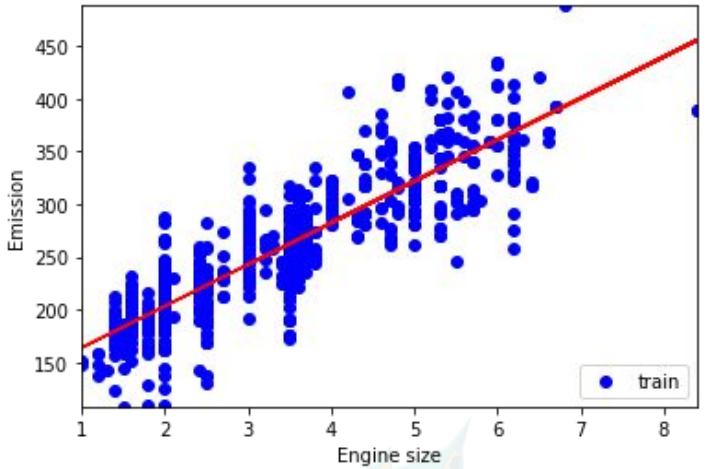
Sumber: DTS - Thematic Academy 2021 - Regresi

Institut Teknologi Bandung www.itb.ac.id @itb1920 @itbofficial

Simple Linear Regression: Training

```
from sklearn import linear_model
regr = linear_model.LinearRegression()
train_x = np.asanyarray(train[['ENGINESIZE']])
train_y = np.asanyarray(train[['CO2EMISSIONS']])
regr.fit (train_x, train_y)
# The coefficients
print ('Coefficients: ', regr.coef_)
print ('Intercept: ',regr.intercept_)
```

```
Coefficients:  [[39.43522758]]
Intercept:  [124.41641136]
```



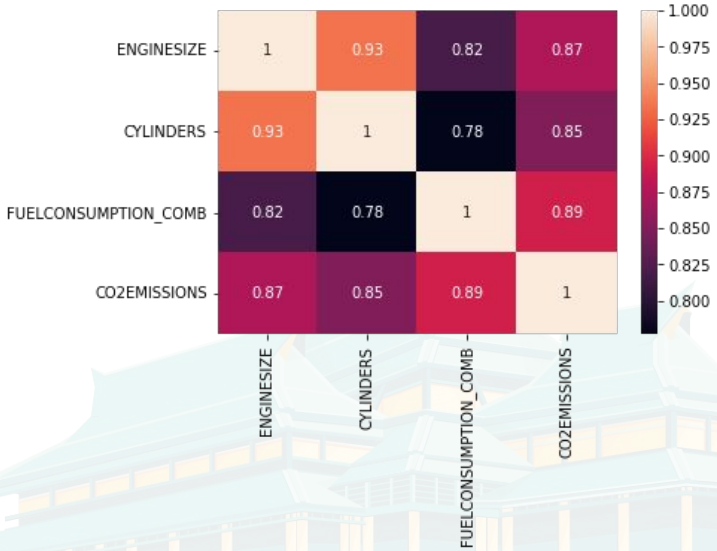
y=125.49+39.14x

Multivariate Linear Regression: Training & Testing

```
from sklearn import linear_model
regr2 = linear_model.LinearRegression()
train_x = np.asanyarray(train[['ENGINESIZE','CYLINDERS','FUELCONSUMPTION_COMB']])
train_y = np.asanyarray(train[['CO2EMISSIONS']])
regr2.fit (train_x, train_y)
print ('Coefficients: ', regr2.coef_)
print ('Intercept: ',regr2.intercept_)
```

```
Coefficients:  [[11.36694334  7.26349823  9.58219844]]
Intercept:  [64.98474043]
```

	ENGINESIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244



Estimating Parameters: Least Square Estimator

- Objective function: minimize sum of the squares of the residuals/errors (SSE)

$$SSE = \sum_{i=1}^{n} e_i^2 = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = \sum_{i=1}^{n} (y_i - b_0 - b_1 x_i)^2$$

$$\frac{\partial (SSE)}{\partial b_0} = -2 \sum_{i=1}^{n} (y_i - b_0 - b_1 x_i)$$

$$\frac{\partial (SSE)}{\partial b_1} = -2 \sum_{i=1}^{n} (y_i - b_0 - b_1 x_i) x_i$$

$$n b_0 + b_1 \sum_{i=1}^{n} x_i = \sum_{i=1}^{n} y_i$$

$$b_0 = \frac{\sum_{i=1}^{n} y_i - b_1 \sum_{i=1}^{n} x_i}{n} = \bar{y} - b_1 \bar{x}$$

$$b_1 = \frac{n \sum_{i=1}^{n} x_i y_i - \left(\sum_{i=1}^{n} x_i \right) \left(\sum_{i=1}^{n} y_i \right)}{n \sum_{i=1}^{n} x_i^2 - \left(\sum_{i=1}^{n} x_i \right)^2} = \frac{\sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n} (x_i - \bar{x})^2} = \sigma_{XY} / \sigma^2_X$$

Institut Teknologi Bandung www.itb.ac.id @itb1920 @itbofficial

Let X be a random variable with probability distribution $f(x)$ and mean μ. The variance of X is

$$\sigma^2 = E[(X-\mu)^2] = \sum_x (x-\mu)^2 f(x), \qquad \text{if } X \text{ is discrete, and}$$

$$\sigma^2 = E[(X-\mu)^2] = \int_{-\infty}^{\infty} (x-\mu)^2 f(x)\, dx, \qquad \text{if } X \text{ is continuous.}$$

The positive square root of the variance, σ, is called the **standard deviation** of X.

Let X and Y be random variables with joint probability distribution $f(x,y)$. The covariance of X and Y is

$$\sigma_{XY} = E[(X-\mu_X)(Y-\mu_Y)] = \sum_x \sum_y (x-\mu_X)(y-\mu_y) f(x,y)$$

if X and Y are discrete, and

$$\sigma_{XY} = E[(X-\mu_X)(Y-\mu_Y)] = \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} (x-\mu_X)(y-\mu_y) f(x,y)\, dx\, dy$$

if X and Y are continuous.

Let X and Y be random variables with covariance σ_{XY} and standard deviations σ_X and σ_Y, respectively. The correlation coefficient of X and Y is

$$\rho_{XY} = \frac{\sigma_{XY}}{\sigma_X \sigma_Y}.$$

Institut Teknologi Bandung www.itb.ac.id @itb1920 @itbofficial

Implement From Scratch: Simple Linear Regression

From Scratch

b1=covariance(X,Y)/variance(X)
b0=mean(Y)-b1*mean(X)

```
# Coefficients:  [[39.43522758]]
# Intercept:  [124.41641136]
print('cov(X,Y)=',train.CO2EMISSIONS.cov(train.ENGINESIZE))
print('var(X)=',train.ENGINESIZE.var())
b1=train.CO2EMISSIONS.cov(train.ENGINESIZE)/train.ENGINESIZE.var()
b0=train.CO2EMISSIONS.mean()-b1*train.ENGINESIZE.mean()
print('Coefficient=',b1)
print('Intercept=',b0)
```

```
cov(X,Y)= 78.95604822766053
var(X)= 2.0021704722919877
Coefficient= 39.43522757943557
Intercept= 124.41641136306387
```

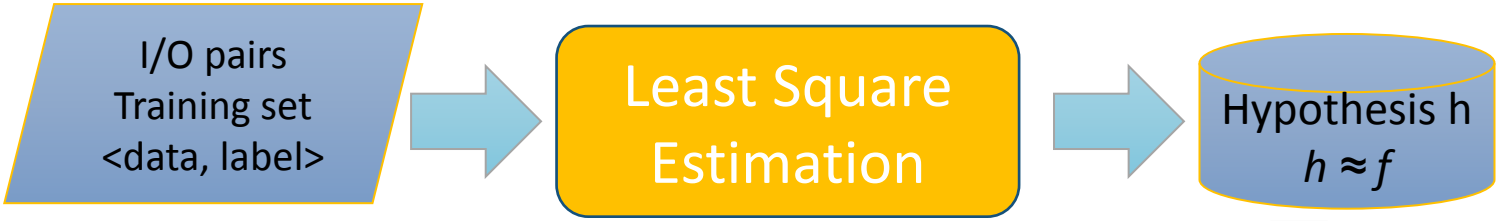
```
from sklearn import linear_model
regr = linear_model.LinearRegression()
train_x = np.asanyarray(train[['ENGINESIZE']])
train_y = np.asanyarray(train[['CO2EMISSIONS']])
regr.fit (train_x, train_y)
# The coefficients
print ('Coefficients: ', regr.coef_)
print ('Intercept: ',regr.intercept_)
```

```
Coefficients:  [[39.43522758]]
Intercept:  [124.41641136]
```


Institut Teknologi Bandung www.itb.ac.id @itb1920 @itbofficial

Linear Regression: Supervised Learning

Learning a (possibly incorrect) general function from specific input-output pairs



	ENGINESIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244



ENGINESIZE	CYLINDERS	FUELCONSUMPTION_COMB
2.0	4	8.5
2.4	4	9.6
1.5	4	5.9

```
test_y[0:3]
```

```
array([[196],
       [221],
       [136]],
```

y=64.98+11.37x1
+7.26x2+9.58x3

```
#test_y_ = regr2.predict(test_x)
test_y_[0:3]
```

```
array([[198.22130673],
       [213.30850235],
       [167.62411912]])
```

Linear Regression: Testing

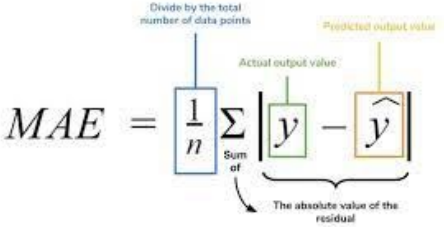
```
from sklearn.metrics import r2_score

test_x = np.asanyarray(test[['ENGINESIZE']])
test_y = np.asanyarray(test[['CO2EMISSIONS']])
test_y_ = regr.predict(test_x)

print("Mean absolute error: %.2f" % np.mean(np.absolute(test_y_ - test_y)))
print("Residual sum of squares (MSE): %.2f" % np.mean((test_y_ - test_y) ** 2))
print("R2-score: %.2f" % r2_score(test_y_ , test_y) )
```

Univariate: y=125.49+39.14x

```
Mean absolute error: 22.50
Residual sum of squares (MSE): 840.97
R2-score: 0.73
```



$$MAE = \frac{1}{n}\sum \left| y - \hat{y} \right|$$

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n} (y_i - \tilde{y}_i)^2$$

Multivariate: y=64.98+11.37x1+7.26x2+9.58x3

```
from sklearn.metrics import r2_score

test_x = np.asanyarray(test[['ENGINESIZE','CYLINDERS','FUELCONSUMPTION_COMB']])
test_y_ = regr2.predict(test_x)

print("Mean absolute error: %.2f" % np.mean(np.absolute(test_y_ - test_y)))
print("Residual sum of squares (MSE): %.2f" % np.mean((test_y_ - test_y) ** 2))
print("R2-score: %.2f" % r2_score(test_y_ , test_y) )
```

```
Mean absolute error: 15.20
Residual sum of squares (MSE): 443.90
R2-score: 0.87
```

$$R2 = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y})},$$

Institut Teknologi Bandung www.itb.ac.id @itb1920 @itbofficial

Linear Regression - LSE: Summary

Target function is estimated by **hypothesis** as a linear function of X:

$h_{\boldsymbol{\theta}}(x)=\boldsymbol{\theta}_0+\boldsymbol{\theta}_1 x_1+\ldots+\boldsymbol{\theta}_n x_n=\sum \boldsymbol{\theta}_i x_i$, i in [0..n]

$\rightarrow h_{\boldsymbol{\theta}}(x) = \boldsymbol{\theta}^T.\mathbf{x}$

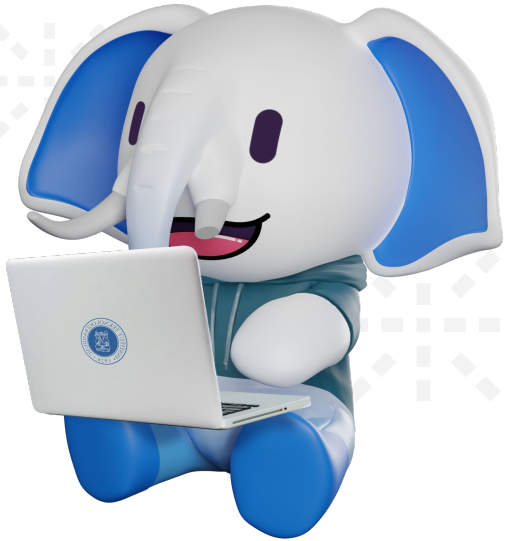
Cost function $J(\boldsymbol{\theta})$ assigns a large cost to bad predictions h(x) and small cost to good predictions h(x). For LSE, we used **sum of squares errors (SSE)**.

Find good $\boldsymbol{\theta}$ (parameters or weights): $\boldsymbol{\theta}^*=\text{argmin } J(\boldsymbol{\theta})$

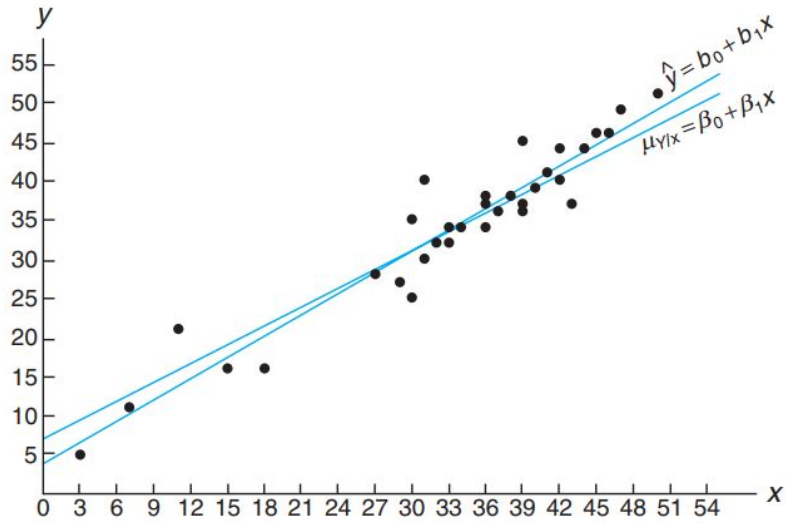
Analytical solution: find $\boldsymbol{\theta}$ that solve $\nabla_J(\boldsymbol{\theta})=0$.

If $\nabla_J(\boldsymbol{\theta})=0$ is unable to solve, use iterative numerical methods. The simplest method is gradient descent.

Logistic Regression



Regression Analysis



Walpole, R. E., Myers, R. H., Myers, S. L., & Ye, K. (2012). Probability and Statistics for engineering and sciences. *Pearson Education*, 430-435.

Finding the best relationship between *Y* and *x:* not deterministic, random error

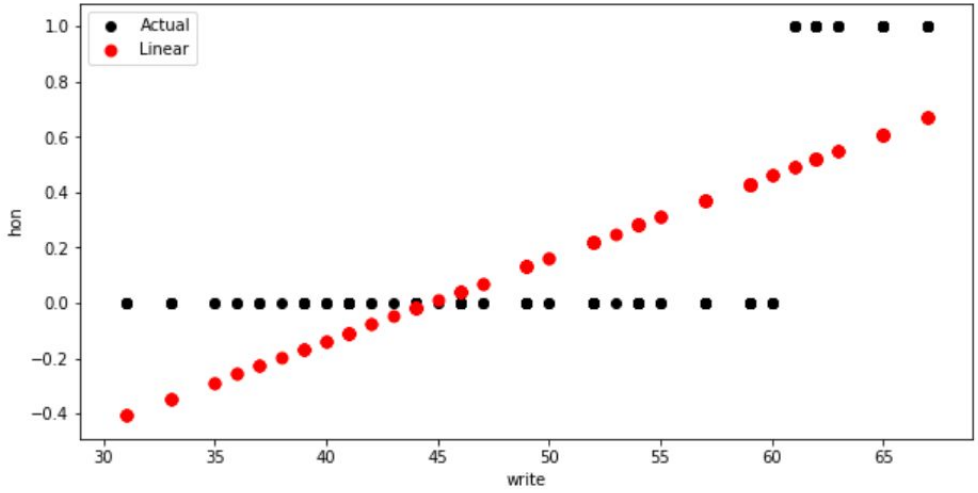
Quantifying the strength of that relationship. Random error with E(error)=0 and homoscedasticity. Least Squared Estimation (LSE)

Predicting of the response value y given values of the regressor *x*.

Institut Teknologi Bandung www.itb.ac.id @itb1920 @itbofficial

Classification Using Linear Regression

Example: Dataset has 200 observations (5 attributes), and the target attribute is **hon**, indicating if a student is in an honors class or not.



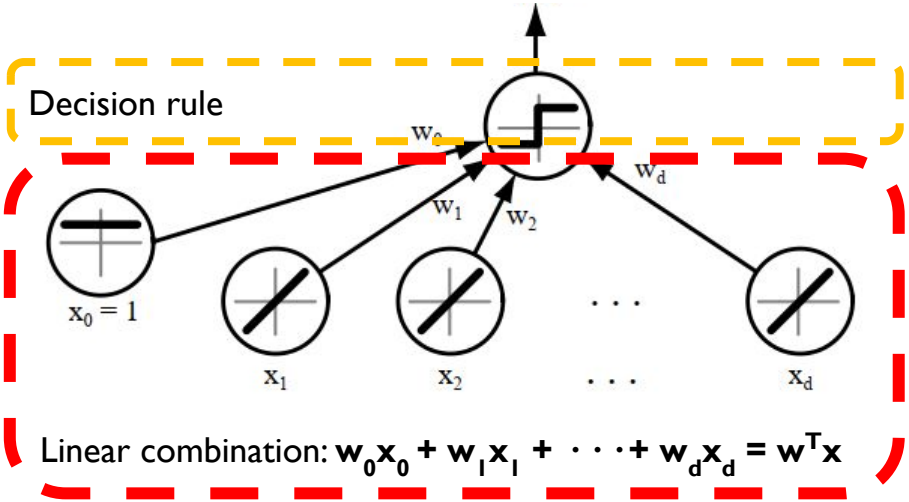
We are able to predict the value along the Y-axis. If Y is greater than 0.5 (above the green line), predict that the student is in honor class otherwise not in honor class.

Linear Model or
Linear Discriminant Function or
Linear Classifier

Institut Teknologi Bandung www.itb.ac.id @itb1920 @itbofficial

Linear Discriminant Function

Linear Combination + Decision Rule



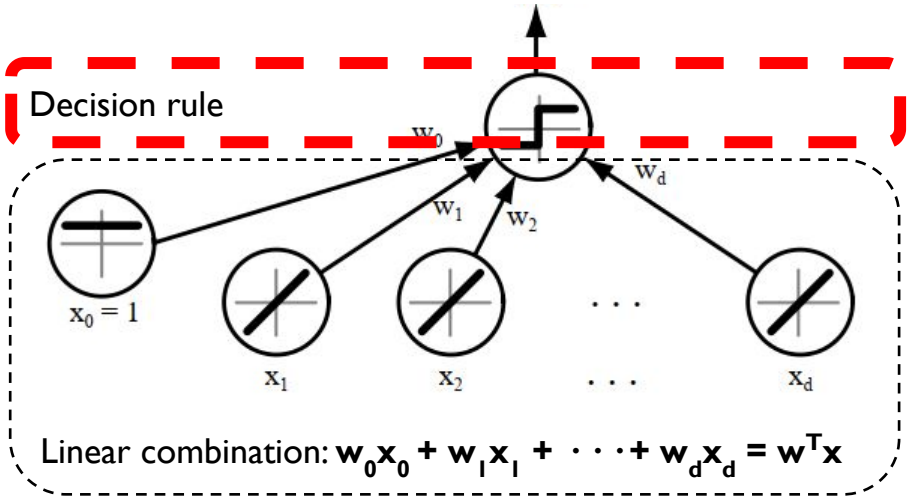
Discriminant function:
a linear combination of the components of x

$$g(\mathbf{x}) = w_0x_0 + w_1x_1 + \cdots + w_dx_d = \mathbf{w}^T\mathbf{x}$$

RO Duda, PE Hart, and DG Stork, Pattern Classification, 2nd edition, John Wiley & Sons, 2001
https://www.cs.toronto.edu/~rgrosse/courses/csc321_2018/readings/L03%20Linear%20Classifiers.pdf

Institut Teknologi Bandung www.itb.ac.id @itb1920 @itbofficial

Hyperplane Decision Surface



Decision rule:

Decide ω_1 if $g(x)>0$ and ω_2 if $g(x)<0$

$g(x)=0$: decision surface

ω_1 and ω_2 are target classes

RO Duda, PE Hart, and DG Stork, Pattern Classification, 2nd edition, John Wiley & Sons, 2001

Institut Teknologi Bandung www.itb.ac.id @itb1920 @itbofficial

Hyperplane Decision Surface

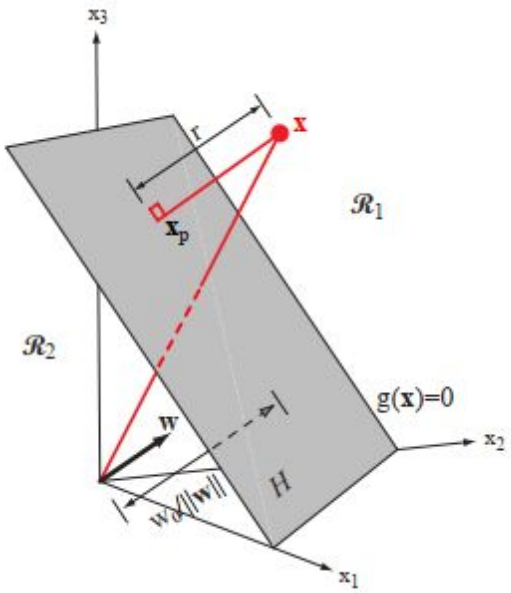


Figure 5.2: The linear decision boundary H, where $g(\mathbf{x}) = \mathbf{w}^t\mathbf{x} + w_0 = 0$, separates the feature space into two half-spaces $\mathcal{R}_1$ (where $g(\mathbf{x}) > 0$) and $\mathcal{R}_2$ (where $g(\mathbf{x}) < 0$).

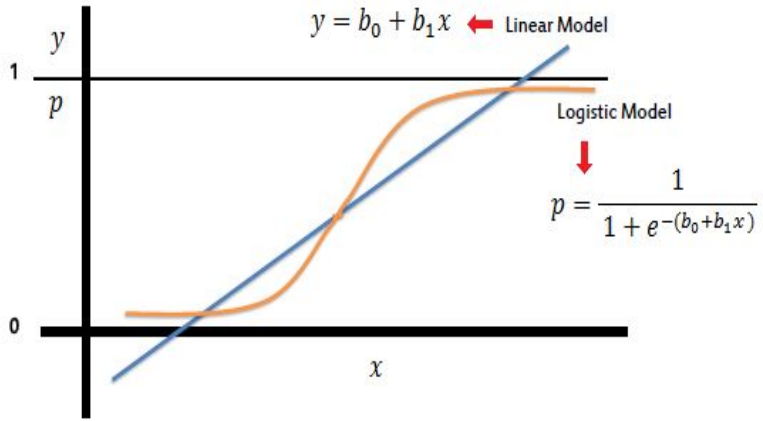
RO Duda, PE Hart, and DG Stork, Pattern Classification, 2nd edition, John Wiley & Sons, 2001

Discriminant function:
$g(\mathbf{x})=\mathbf{w}^T\mathbf{x}$

Decision rule:
Decide ω_1 if $g(x)>0$ and ω_2 if $g(x)<0$
$g(x)=0$: decision surface
ω_1 and ω_2 are target classes

Institut Teknologi Bandung www.itb.ac.id @itb1920 @itbofficial

Linear Regression	Logistic Regression
Least squares	Maximum likelihood
Constant variance	Non-constant variance
Response variable is normal	Response variable is binary
	Estimate the probability of classes



http://www.saedsayad.com/logistic_regression.htm

Logistic (sigmoid) function

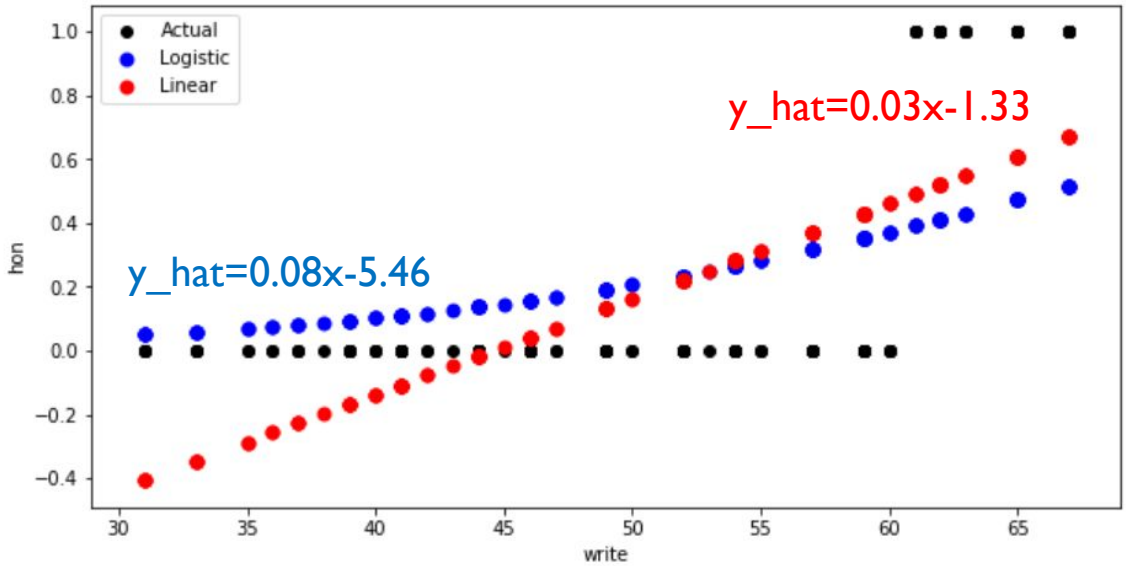
$$f(x) = \frac{1}{1 + e^{-x}}$$

Walpole, R. E., Myers, R. H., Myers, S. L., & Ye, K. (2012). Probability and Statistics for engineering and sciences. *Pearson Education*, 430-435.

Institut Teknologi Bandung www.itb.ac.id @itb1920 @itbofficial

Logistic Regression

$$\hat{y} = \log\left(\frac{p}{1-p}\right) = \boldsymbol{b}^T\boldsymbol{x} = b_0.1 + b_1x_1 + \cdots + b_dx_d$$



Walpole et al. (2012):
Odds of success = p/(1-p)
Logistic regression estimates **the probability of classes:**

$$p = P(y = 1|\boldsymbol{x}, \boldsymbol{b}) = \frac{1}{1 + e^{-\boldsymbol{b}^T\boldsymbol{x}}}$$

Coefficients in logistic regression are in terms of the **log odds**, that is, the coefficient 0.08 implies that a one unit change in "write" results in a 0.08 unit change in the log of the odds.

https://stats.idre.ucla.edu/stata/faq/how-do-i-interpret-odds-ratios-in-logistic-regression/

Institut Teknologi Bandung www.itb.ac.id @itb1920 @itbofficial

Summary: Logistic Regression

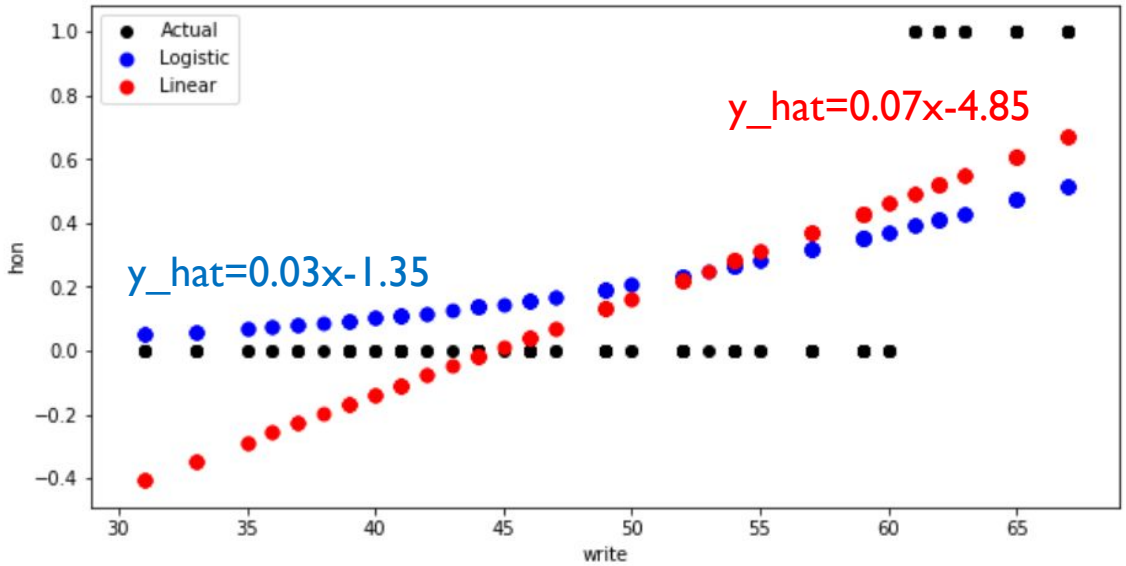
- Linear model
- Binary classification
- Soft classification

$$\hat{y} = \log\left(\frac{p}{1-p}\right) = \boldsymbol{b}^{\boldsymbol{T}}\boldsymbol{x} = b_0.1 + b_1x_1 + \cdots + b_dx_d$$

$$p = P(y = 1|\boldsymbol{x}, \boldsymbol{b}) = \frac{1}{1 + e^{-\boldsymbol{b}^{\boldsymbol{T}}\boldsymbol{x}}}$$

Exercise

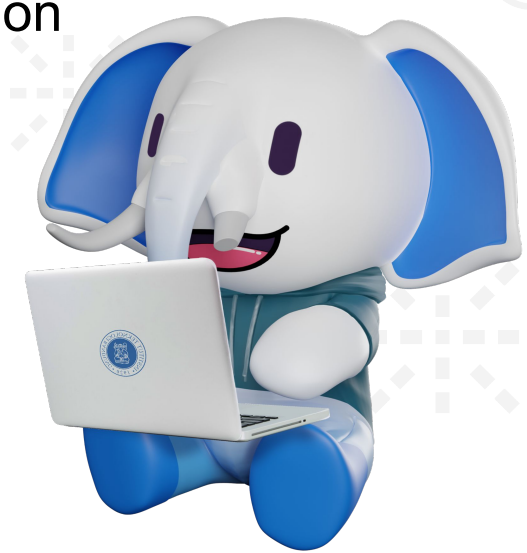




Given these linear regression and logistic regression models for classification, determine whether a student that has write score 65 is in honors class.

Stochastic Gradient Ascent

Optimization Algorithm for Logistic Regression



Logistic Regression

$$\hat{y} = \log\left(\frac{p}{1-p}\right) = \boldsymbol{b^T x} = b_0.1 + b_1 x_1 + \cdots + b_d x_d$$

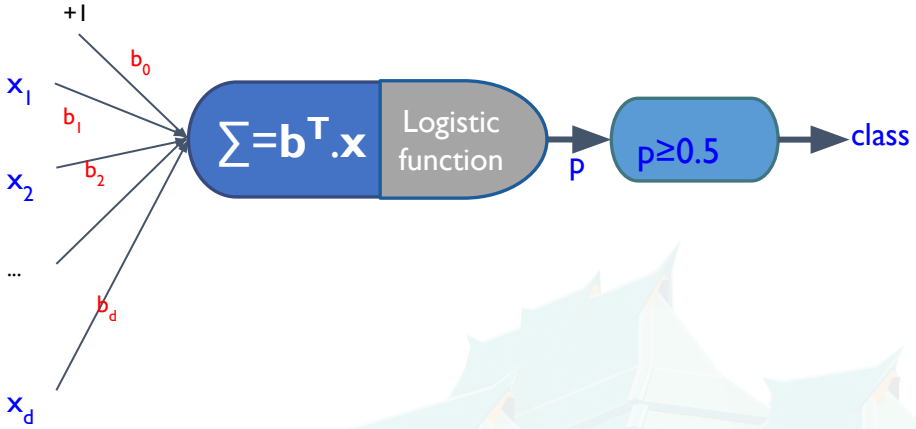
$$p = P(y = 1|\boldsymbol{x}, \boldsymbol{b}) = \frac{1}{1 + e^{-\boldsymbol{b^T x}}}$$

Input $\mathbf{x}=(1, x_1, x_2, \ldots, x_d)$

Model $\mathbf{b}=(b_0, b_1, b_2, \ldots, b_d)$

$\sum = \mathbf{b}^T\boldsymbol{x} = b_0.1 + b_1 x_1 + \cdots + b_d x_d$

Output = $\sigma(\sum)$



Maximum Likelihood Estimator For Logistic Regression

Estimator that results in a maximum value for its joint probability or maximizes the likelihood of the sample

Formal definition (Elkan, 2014):

Given the training set $\{<x_1,y_1>..<x_n,y_n>\}$, learn logistic regression classifier by maximizing the log joint conditional likelihood, that is the sum of log conditional likelihood (LCL) for each training example. x_{ij} is the value of the jth feature of the ith training example.

$$LCL = \sum_{i=1}^{n} \log L(\theta; y_i|x_i) = \sum_{i=1;y_i=1}^{n} \log p_i + \sum_{i=1;y_i=0}^{n} \log(1-p_i)$$

$$\rightarrow \frac{\partial LCL}{\partial b_j} = \sum_i (y_i - p_i)x_{ij}$$

Stochastic gradient **ascent** is optimization method that changes the coefficient values (as random approximation to true derivative) to **increase** the log likelihood based on a randomly chosen example at a time.

Stochastic gradient update of b_j is:

η : learning rate

$$b_j = b_j + \eta(y_i - p_i)x_{ij}$$

https://cseweb.ucsd.edu/~elkan/250B/logreg.pdf

Institut Teknologi Bandung www.itb.ac.id @itb1920 @itbofficial

Stochastic Gradient Ascent for Logistic Regression

INPUT: TRAINING DATA D={<X_1,Y_1>...<X_N,Y_N>}; MAX-ITER T; LEARNING RATE η

Initialize **b**

For t=1, …,T:

One iteration = one epoch

- For each example <xi,yi>: #randomly chosen example
  - pi=prediction for xi using the current coefficients **b**
  - For each non-zero feature of xij: $b_j = b_j + \eta(y_i - p_i)x_{ij}$

Return **b**

http://www.cs.carnegie.edu/~wcohen/10-605/sgd-part2.pdf

D:{<[52,41],0>,<[62,58],1>};T=1;η=0.1

b=[0,0,0] #b0=0; b1=0; b2=0

t=1:

<[62,58],1>:

pi=1/(1+e^{-0})=0.5

b0=0+0.1(1-0.5)*1=0.05

b1=0+0.1(1-0.5)*62=3.1

b2=0+0.1(1-0.5)*58=2.9

<[52,41],0>:

pi=1/(1+$e^{-(0.05+3.1*52+2.9*41)}$)

=1/(1+$e^{-280.15}$)=1

b0=0.05+0.1(0-1)*1= -0.05

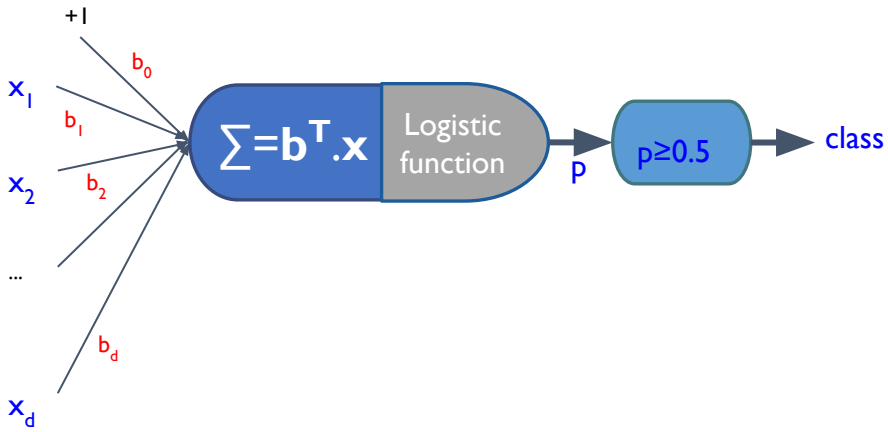
b1=3.1+0.1(0-1)*52= -2.1

b2=2.9+0.1(0-1)*41= -1.2

Prediction

- b0= -0.05; b1= -2.1; b2= -1.2
- D: {<[52,41],0>,<[62,58],1>}
  - **x1**=[52,41]: p1= $1/(1+e^{-(-0.05-2.1*52-1.2*41)}) = 1/(1+e^{158.45}) = 1.53*10^{-69}$ □ class=0 (p1<0.5)
  - **x2**=[62,58]: p2= $1/(1+e^{-(-0.05-2.1*62-1.2*58)}) = 1/(1+e^{199.85}) = 1.61*10^{-87}$ □ class=0 (p2<0.5)
- Akurasi training = ½ = 50%

Summary: Logistic Regression



- Model: $\mathbf{b} \in \Re^{d+1}$
- Maximum Likelihood estimator
- Stochastic gradient ascent

Institut Teknologi Bandung www.itb.ac.id @itb1920 @itbofficial

References

- Walpole, R. E., Myers, R. H., Myers, S. L., & Ye, K. (2012). Probability and Statistics for engineering and sciences. *Pearson Education*, 430-435. Chapter 11 & 12.12, 9.14
- RO Duda, PE Hart, and DG Stork, Pattern Classification, 2nd edition, John Wiley & Sons, 2001. Chapter 5
- Charles Elkan (2014). Maximum Likelihood, Logistic Regression, and Stochastic Gradient Training. https://cseweb.ucsd.edu/~elkan/250B/logreg.pdf
- Russell, S., & Norvig, P. (2010). Artificial intelligence: a modern approach. 3rd edition. Chapter 18.6.4.

Thank You

